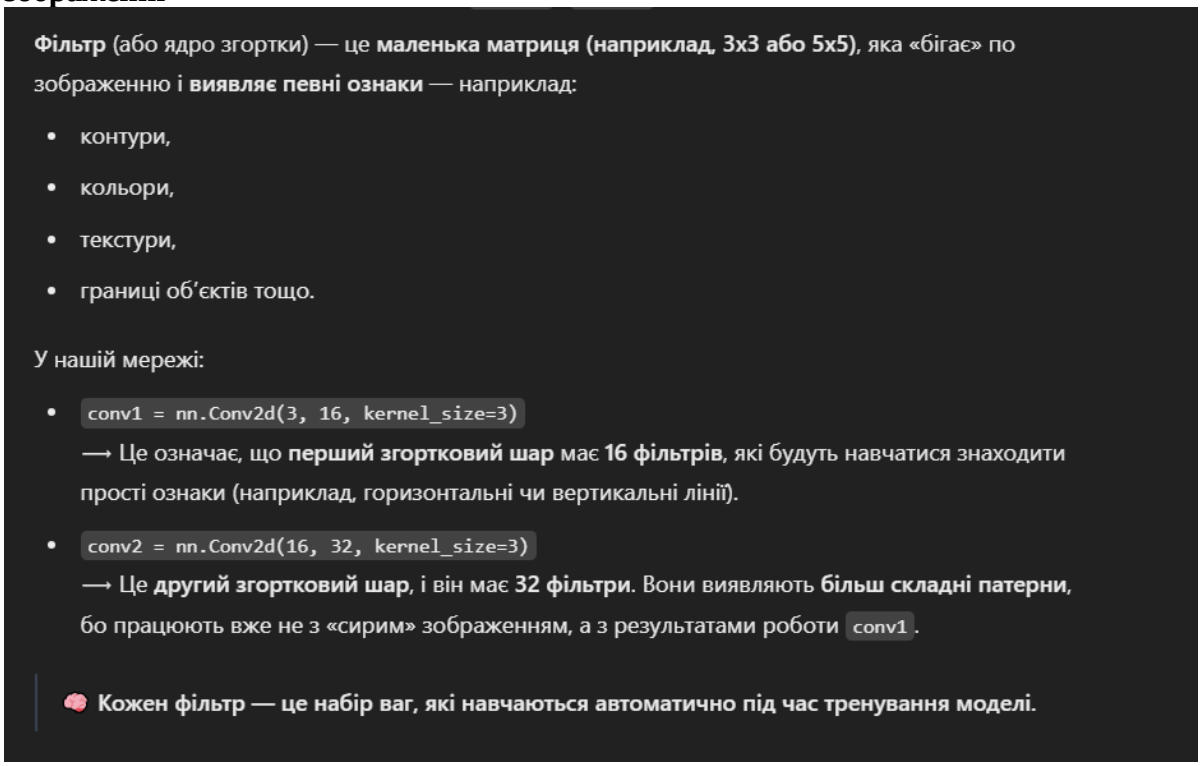
У даній лабораторній роботі необхідно було створити і навчити CNN та RNN.
CNN (Convolutional Neural Network конволушенал неурал нетворк) — для класифікації зображень із набору **CIFAR-10**. Тобто як він працює
1)Спочатку Згорткові шари (Convolution Layers) – знаходять особливості на зображенні



Кожен фільтр шукає щось своє (горизонтальні лінії, вертикальні, кола).
2) Далі йде Шар пулінгу (Pooling Layer) – зменшує розмір зображення, залишаючи головне
3) Далі Повнозв'язні шари (Fully Connected Layers) – приймають рішення, тобто
Після того, як CNN витягла корисну інформацію з картинки, вона "вирівнює" отримані дані у певний список чисел і передає їх у звичайну нейромережу, яка приймає фінальне рішення.
4) Ну і вкінці йде функція активації (ReLU) , вона допомагає моделі приймати рішення
Вона працює як фільтр, який прибирає непотрібні або неважливі значення, щоб мережа краще вчилася.

Значить далі у мене прописаний Train та Test процесс моделі CNN. Тут у нас цикл Train та циклу Test, які виконуються впродовж кількох епох. У кожній епосі модель навчається на тренувальному наборі, після чого оцінюється на тестовому. В ході кожної епохи обчислюється втрата (loss) та точність (accuracy), які зберігаються у відповідні списки для подальшої побудови графіків продуктивності.

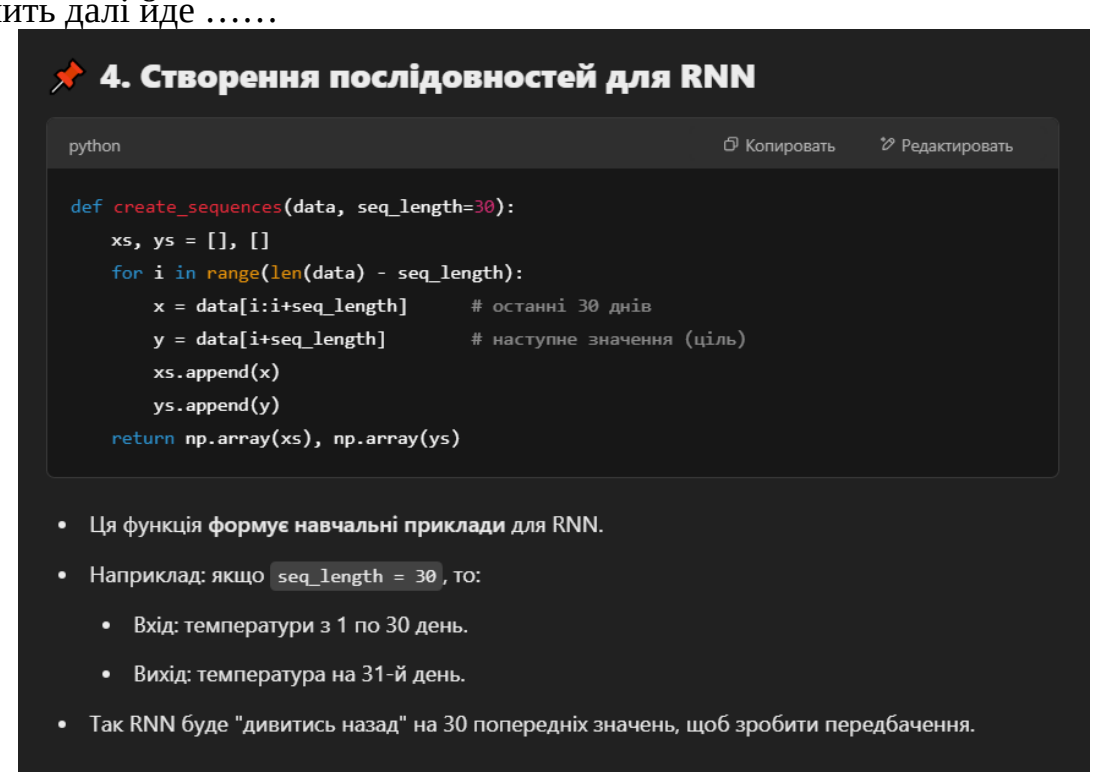
Так, ну і далі власне у мене йдуть дві функції для візуалізації фільтрів (ну власне матриць), та Feature Maps для першого шару (**тобто набір зображень, які показують, що побачили фільтри першого згорткового шару на вході** (**там це краї, кути, текстури**).) і відповідно Feature Maps другого шару, (**тобто це нові ознаки, які побудовані на основі Feature Maps першого шару і вони відображають більш складні структури**)

Значить далі у мене йде реалізація RNN,
RNN (Recurrent Neural Network) — для прогнозування значень у **часовому ряді** (у мене це будуть температурні дані).

1) **Беремо вхідні дані** (наприклад, слово в реченні або значення у часовому ряді).
2) **Передаємо їх у нейрон** (нейрон запам’ятовує контекст).
3) **Рухаємось далі**, додаючи нові дані, але з урахуванням того, що було раніше.

Значить по коду, я спочатку завантажив дані, відповідно зобразив графік даних зміни температур

Значить далі йде ……



Далі йде

- Генерація входів і виходів
- Розділення на тренувальні та тестові дані
- Перетворення у тензори для PyTorch
- Формування DataLoader'ів

Ну відповідно далі йде реалізація власне RNN.

- Значить спочатку йде створення класу RNN моделі
- Після чого прописана Ініціалізація (моделі, функції втрат і оптимізатора)

- Далі прописано кількість епох для навчання, та списки для зберігання Train та Test значень
- Значить далі прописана функція для очищення списку прихованих станів перед кожною епохою (ну відповідно прихований стан це така внутрішня пам’ять рекурентної нейронної мережі, яка зберігає інформацію про попередні елементи послідовності.
- Ну і далі відповідно йде цикл для Train та Test , та відповідно обрахунки втрат. Ну і виведення результатів обрахунків та графіків на екран

Hidden state — це **внутрішня пам’ять** рекурентної нейронної мережі (RNN або LSTM), яка зберігає **інформацію про попередні елементи послідовності**.

Згортка — це математична операція, яка дозволяє **знайти на зображенні важливі ознаки** (наприклад, краї, кути, текстури) за допомогою маленької фільтр-матриці.

Фільтри — це **маленькі матриці (зазвичай 3×3 або 5×5)**, які “шукають” певні патерни на зображенні під час згортки.